

Densest subgraph

Executive summary

1. Greedy peeling algorithm

We implemented the greedy peeling algorithm closely following the lecture notes. In each step, the algorithm computes the current density, removes a vertex of minimum degree, and continues with the remaining graph. The implementation uses the linked-list based bucket structure from the lecture notes, so vertices can be moved between degree buckets efficiently during the peeling process.

To keep the running time linear, we first store all degrees in an array and then keep an array of doubly linked lists, where list k contains all vertices of current degree k . We also store a pointer from every vertex to its position in the correct list, and we maintain a variable k^* for the smallest non-empty degree bucket. This means that when an edge is removed, we only move the neighbor once from one bucket to the next smaller one, and each edge is touched only when it is deleted.

One small challenge is that the algorithm destroys the graph while it runs, but at the end we still need the best subgraph and not only its score. Instead of copying the whole edge set whenever we see a better solution, we store the number of already removed vertices at the best point. After the peeling process is finished, we reconstruct the best vertex set from this removal order. This keeps the implementation simple and avoids losing the linear runtime.

2. Results for the density objective

For the OpenFlights dataset, the density objective gives the following five clusters:

- Cluster 1, $|S| = 180$: a large international hub cluster with many airports from the US and Western Europe.
- Cluster 2, $|S| = 53$: an almost purely Chinese domestic network.
- Cluster 3, $|S| = 29$: a South Asia and Middle East cluster with many airports from India, Pakistan and Saudi Arabia.
- Cluster 4, $|S| = 57$: a more unusual mixed cluster with many Russian and Brazilian airports.
- Cluster 5, $|S| = 18$: a compact Australia and New Zealand cluster.

This shows that the density objective can recover both a very large global hub structure and smaller regional flight networks.

For the Facebook dataset, the density objective also returns several clearly interpretable clusters:

- Cluster 1, $|S| = 324$: mainly US military and government pages, but also some TV show pages. Examples are 101st Airborne Division, 1st Infantry Division and 24: Legacy.
- Cluster 2, $|S| = 89$: EU institutions and official EU pages. Examples are Commission européenne en France, Council of Europe and EU Budget.
- Cluster 3, $|S| = 662$: a very large mixed politics and government cluster with diplomatic pages and politicians from several countries. Examples are Africa Regional Media Hub, American Center New Delhi and Anthony Albanese MP.
- Cluster 4, $|S| = 55$: French ministries and prefectures. Examples are Ministère de la Justice, Police Nationale and Préfecture de Police.
- Cluster 5, $|S| = 311$: another broad politician-heavy cluster with many Australian political and diplomatic pages. Examples are Alan Tudge MP, Andrew Scheer and Australia in India.

So on Facebook, the density objective sometimes gives very clear thematic communities, but in other rounds it prefers larger and looser groups.

3. Results for the Tsourakakis objective

When we changed the objective function, the results became more specific. For OpenFlights, the five clusters are:

- Cluster 1, $|S| = 93$: a smaller Western European core.
- Cluster 2, $|S| = 45$: a mostly US domestic network.
- Cluster 3, $|S| = 55$: again a Chinese domestic cluster.
- Cluster 4, $|S| = 29$: again a South Asia and Middle East cluster.
- Cluster 5, $|S| = 22$: a mostly Brazilian domestic group.

The modified objective therefore keeps some of the same regional structures, but it removes many airports from the broader international clusters and keeps only tighter subsets.

On Facebook, the difference was even stronger. The five Tsourakakis clusters are:

- Cluster 1, $|S| = 60$: only TV shows. Examples are Brooklyn Nine-Nine, Family Guy and Empire.
- Cluster 2, $|S| = 129$: US Army and related military institutions. Examples are 101st Airborne Division, 3rd Infantry Division and Aberdeen Proving Ground.
- Cluster 3, $|S| = 89$: again a cluster of EU institutions. Examples are Council of the European Union, EU Budget and EU Environment.
- Cluster 4, $|S| = 38$: South Korean government pages. Examples are Ministry of Science and ICT, Ministry of Health and Welfare, Republic of Korea, and 대한민국 법무부.
- Cluster 5, $|S| = 41$: mainly Australian Labor politicians. Examples are Anthony Albanese MP, Bill Shorten MP and Julia Gillard.

So in our results, the density objective tends to return larger and broader groups, while the Tsourakakis objective tends to return smaller and more clique-like communities. This difference is visible on both datasets, but especially on Facebook.

Basic Graph Class

We already provide the basic implementation of a graph class.

Iterating over nodes can be done by using `for u in G.nodesToEdges` and iterating over the edges incident upon node `u` can be done using `for v in G.nodesToEdges[u]`. To check whether an edge (u,v) exists use `G.edgeExists(u,v)`.

Graphs can be read from files using the `readFromFile` -procedure. Using the optional argument `verticesToIgnore` which expects a `set()` as input, you may exclude some nodes from the graph, i.e., if `verticesToIgnore` is some set S then it will load the graph $G=(V\setminus S, E[V\setminus S])$. Note that you might have to delete header-rows from some data files to make the procedure work.

```
In [24]: class Graph:
    def __init__(self):
        self.numNodes = 0
        self.numEdges = 0

        self.edges = set()
        self.nodesToEdges = {}

    def addEdge(self,u,v):
        if u == v or v in self.nodesToEdges and u in self.nodesToEdges[v]:
            return

        self.numEdges += 1
        self.addNeighbor(u,v)
        self.addNeighbor(v,u)
        self.edges.add((u,v)) # we only add one pair (u,v) and not (v,u)

    def removeEdge(self,u,v):
        self.numEdges -= 1
        self.nodesToEdges[u].remove(v)
        self.nodesToEdges[v].remove(u)

        if (u,v) in self.edges:
            self.edges.remove((u,v))
        else: # (v,u) in self.edges
            self.edges.remove((v,u))

    def addNeighbor(self,u,v):
        if u not in self.nodesToEdges:
            self.numNodes += 1
            self.nodesToEdges[u] = set()

        self.nodesToEdges[u].add(v)

    def edgeExists(self, u, v):
        return ((u,v) in self.edges or (v,u) in self.edges)
```

```

def degree(self, u):
    return len(self.nodesToEdges[u])

def readFromFile(self, filePath, separator=',', verticesToIgnore=set()):
    with open(filePath, 'r') as f:
        for line in f:
            split = line.split(separator)
            u = int(split[0])
            v = int(split[1])

            if u in verticesToIgnore or v in verticesToIgnore:
                continue

            self.addEdge(u, v)

print(f'Finished reading graph with {self.numNodes} nodes and {self.numEdges} edges')

```

Basic Linked List Data Structure

First, implement your own linked list data structure `LinkedList` in which each element is from the class `LinkedListElement`.

```

In [25]: class LinkedListElement:
def __init__(self, id, prevElement=None, nextElement=None):
    self.id = id
    self.prevElement = prevElement
    self.nextElement = nextElement

class LinkedList:
def __init__(self):
    self.head = None
    self.tail = None
    self.size = 0

def appendElement(self, element: LinkedListElement):
    if self.head is None:
        self.head = element
        self.tail = element
    else:
        element.prevElement = self.tail
        self.tail.nextElement = element
        self.tail = element
    self.size += 1

def removeElement(self, element: LinkedListElement):
    if element == self.head:
        self.head = element.nextElement

    if element == self.tail:
        self.tail = element.prevElement

    if element.prevElement:
        element.prevElement.nextElement = element.nextElement
    if element.nextElement:

```

```

        element.nextElement.prevElement = element.prevElement

        element.nextElement = None
        element.prevElement = None
        self.size -= 1

    def pop(self):
        if not self.tail:
            return None
        node_to_remove = self.tail
        self.removeElement(node_to_remove)
        return node_to_remove

```

Implementation of the Greedy Peeling Algorithm

Next, implement the greedy peeling algorithm. The function should be callable using `densestSubgraphGreedyPeeling(G)` where `G` is a graph using the graph class from above. Besides `G`, the function may accept more optional arguments.

```

In [26]: def objectiveValue(num_edges, num_nodes, objective='density'):
    match objective:
        case 'density':
            return num_edges / num_nodes
        case 'tsourakakis':
            return num_edges - (num_nodes * (num_nodes - 1)) / 6
        case 'clique_likeness':
            if num_nodes < 2:
                return 0
            return num_edges / (num_nodes * (num_nodes - 1) / 2)
        case _:
            raise ValueError(f'Unknown objective: {objective}')

    def densestSubgraphGreedyPeeling(G, objective='density'):
        max_id = max(G.nodesToEdges) # needed to allow non-continuous labeling
        d = [0] * (max_id + 1)
        # im assuming here that the vertices are named 0 - n <-- now also fine with a f
        for e in G.edges:
            d[e[0]] += 1
            d[e[1]] += 1

        L = list()
        for _ in range(0, G.numNodes):
            L.append(LinkedList())

        k_star = float('inf')
        pointers_to_vertices = [None] * (max_id + 1) # needed to allow non-continuous
        #for v, degree in enumerate(d):
        for v in G.nodesToEdges: # works with a non-continuous degree list
            degree = d[v]
            element = LinkedListElement(v)
            L[degree].appendElement(element)
            pointers_to_vertices[v] = element

```

```

        if degree < k_star:
            k_star = degree

    #best_S = G.edges.copy() # O(m)
    best_num_edges = G.numEdges
    best_score = objectiveValue(G.numEdges, G.numNodes, objective)
    removed_vertices = []
    best_removed_count = 0
    num_nodes = G.numNodes
    while num_nodes != 1:
        v_star = L[k_star].pop()
        removed_vertices.append(v_star.id)
        neighbors = G.nodesToEdges[v_star.id].copy() # O(m)
        for u in neighbors:
            G.removeEdge(v_star.id, u)
            current = pointers_to_vertices[u]
            L[d[current.id]].removeElement(current) # remove from L[d[u]]
            d[current.id] -= 1
            L[d[current.id]].appendElement(current) # add to L[d[u]-1]

            if d[current.id] < k_star:
                k_star = d[current.id]
        num_nodes -= 1

    score = objectiveValue(G.numEdges, num_nodes, objective)
    if score > best_score:
        best_num_edges = G.numEdges
        best_score = score
        #best_S = G.edges.copy() # O(m) <- we are inside a O(n) loop, this would be O(n^2)
        best_removed_count = len(removed_vertices)

    while k_star < len(L) and L[k_star].size <= 0:
        k_star = (k_star + 1) % G.numNodes

    best_S = set(removed_vertices[best_removed_count:]) # "get back" all vertices that were removed
    best_S.add(L[k_star].head.id) # also add last vertex still in the list

    return best_S, best_num_edges, best_score

def runDisjointSubgraphExtraction(filePath, objective='density'):
    verticesToIgnore = set()
    results = []

    for round_idx in range(1, 6):
        working_graph = Graph()
        working_graph.readFromFile(filePath, ',', verticesToIgnore)
        vertex_set, num_edges, score = densestSubgraphGreedyPeeling(working_graph,

        results.append({
            'round': round_idx,
            'vertex_set': set(vertex_set),
            'num_vertices': len(vertex_set),
            'num_edges': num_edges,
            'score': score,
        })

```

```

        verticesToIgnore.update(vertex_set)

    return results

#testGraph = Graph()
#testGraph.addEdge(0,1)
#testGraph.addEdge(1,2)
#testGraph.addEdge(1,3)
#testGraph.addEdge(1,25)
#testGraph.addEdge(2,3)
#testGraph.addEdge(2,25)
#testGraph.addEdge(3,25)
#densestSubgraphGreedyPeeling(testGraph)

```

Experiments for Densest Subgraph

Now, the experiments for the two datasets follow.

Experiments for the OpenFlights Dataset

```

In [27]: openflights_disjoint_subgraphs = runDisjointSubgraphExtraction('data/openflights_ed

for result in openflights_disjoint_subgraphs:
    print(f"Round {result['round']}:")
    print(f"|S| = {result['num_vertices']}, density = {result['score']}\n")

```

Finished reading graph with 3214 nodes and 18858 edges.

Finished reading graph with 2643 nodes and 7349 edges.

Finished reading graph with 2513 nodes and 5716 edges.

Finished reading graph with 2415 nodes and 4953 edges.

Finished reading graph with 2234 nodes and 4027 edges.

Round 1:

|S| = 180, density = 25.377777777777776

Round 2:

|S| = 53, density = 14.754716981132075

Round 3:

|S| = 29, density = 7.482758620689655

Round 4:

|S| = 57, density = 5.385964912280702

Round 5:

|S| = 18, density = 4.611111111111111

AI-Assisted OpenFlights Visualizations

The following cell was written with AI assistance. It creates one overview world map for all five OpenFlights clusters and five additional cluster-specific maps that also display the intra-

cluster flight edges. The figures are shown in the notebook and saved as PNG files in the `generated/` directory.

```
In [28]: # AI-assisted visualization code.
# It creates one overview map for all clusters and one route map for each cluster.

import csv
from IPython.display import Image, Markdown, display
import os
import plotly.graph_objects as go

with open('data/openflights/nodes.csv', newline='', encoding='utf-8') as f:
    reader = csv.reader(f)
    next(reader)
    openflights_nodes = {
        int(row[0]): {
            'name': row[2],
            'city': row[3],
            'country': row[4],
            'lat': float(row[7]),
            'lon': float(row[8]),
        }
        for row in reader
    }

openflights_edges = set()
with open('data/openflights_edges_cleaned.csv', newline='', encoding='utf-8') as f:
    for row in csv.reader(f):
        u = int(row[0])
        v = int(row[1])
        if u == v:
            continue
        openflights_edges.add((u, v) if u < v else (v, u))

cluster_colors = ['#d1495b', '#edae49', '#66a182', '#2e86ab', '#5c4d7d']

def airport_hover_text(round_idx, node):
    return f"Round {round_idx}<br>{node['name']} ({node['city']}, {node['country']})"

def cluster_nodes(vertex_set):
    node_ids = sorted(vertex_set)
    return node_ids, [openflights_nodes[node_id] for node_id in node_ids]

def cluster_edge_coordinates(vertex_set):
    vertex_set = set(vertex_set)
    lon = []
    lat = []
    edge_count = 0
    for u, v in openflights_edges:
        if u in vertex_set and v in vertex_set:
            lon.extend([openflights_nodes[u]['lon'], openflights_nodes[v]['lon'], N
            lat.extend([openflights_nodes[u]['lat'], openflights_nodes[v]['lat'], N
            edge_count += 1
    return lon, lat, edge_count
```

```

def padded_range(values, lower_bound, upper_bound):
    minimum = min(values)
    maximum = max(values)
    span = maximum - minimum
    if span > 300:
        return None
    padding = max(5, span * 0.15)
    return [max(lower_bound, minimum - padding), min(upper_bound, maximum + padding)]

def apply_geo_style(fig, title, lon_range=None, lat_range=None):
    geo_options = dict(
        projection_type='natural earth',
        showland=True,
        landcolor='#efefef',
        showcountries=True,
        countrycolor='ffffff',
        showocean=True,
        oceancolor='dcefff',
        showcoastlines=True,
        coastlinecolor='#888888',
    )
    if lon_range is not None:
        geo_options['lonaxis'] = dict(range=lon_range)
    if lat_range is not None:
        geo_options['lataxis'] = dict(range=lat_range)
    fig.update_geos(**geo_options)
    fig.update_layout(
        title=title,
        legend_title_text='Cluster',
        width=1200,
        height=700,
        margin=dict(l=20, r=20, t=60, b=20),
    )

os.makedirs('generated', exist_ok=True)

overview_fig = go.Figure()
for result, color in zip(openflights_disjoint_subgraphs, cluster_colors):
    _, nodes = cluster_nodes(result['vertex_set'])
    overview_fig.add_trace(go.Scattergeo(
        lon=[node['lon'] for node in nodes],
        lat=[node['lat'] for node in nodes],
        text=[airport_hover_text(result['round'], node) for node in nodes],
        mode='markers',
        marker=dict(size=7, color=color, opacity=0.85, line=dict(width=0.5, color='black')),
        name=f"Round {result['round']} (|S|={result['num_vertices']})",
        hovertemplate="%{text}<extra></extra>",
    ))

apply_geo_style(overview_fig, 'OpenFlights Dense Subgraphs on a World Map')
overview_path = os.path.join('generated', 'openflights_clusters_world_map.png')
overview_fig.write_image(overview_path, width=1200, height=700, scale=2)
display(Image(filename=overview_path, width=1000))
print(f"Saved overview map to {overview_path}")

for result, color in zip(openflights_disjoint_subgraphs, cluster_colors):

```

```

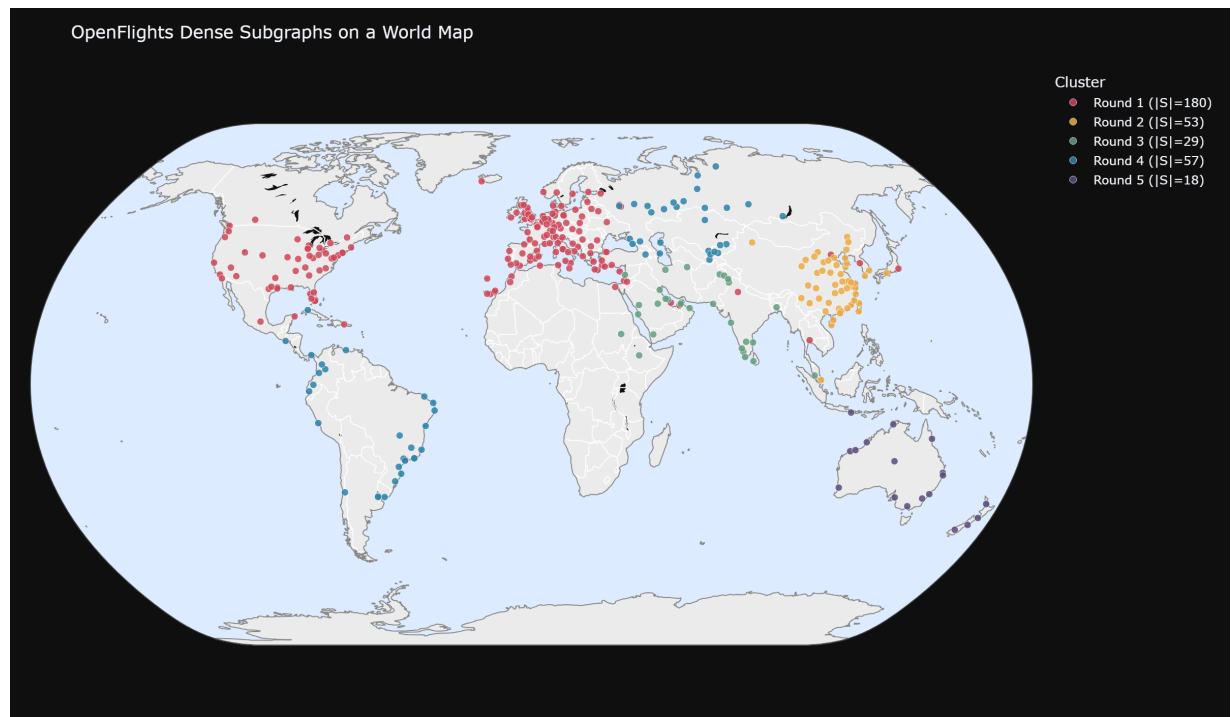
node_ids, nodes = cluster_nodes(result['vertex_set'])
edge_lon, edge_lat, edge_count = cluster_edge_coordinates(node_ids)

cluster_fig = go.Figure()
cluster_fig.add_trace(go.Scattergeo(
    lon=edge_lon,
    lat=edge_lat,
    mode='lines',
    line=dict(width=1, color=color),
    opacity=0.18,
    hoverinfo='skip',
    showlegend=False,
))
cluster_fig.add_trace(go.Scattergeo(
    lon=[node['lon'] for node in nodes],
    lat=[node['lat'] for node in nodes],
    text=[airport_hover_text(result['round'], node) for node in nodes],
    mode='markers',
    marker=dict(size=8, color=color, opacity=0.9, line=dict(width=0.6, color='w
    name=f"Round {result['round']} airports",
    hovertemplate="%{text}<extra></extra>",
))

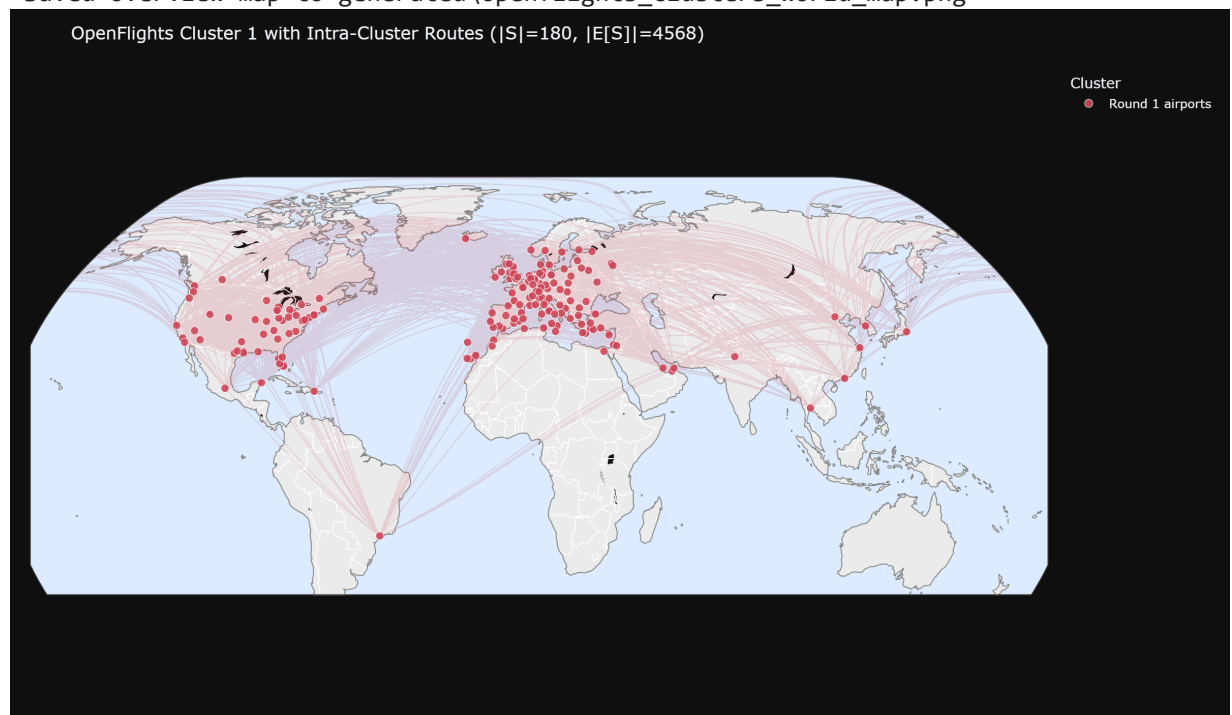
lon_range = padded_range([node['lon'] for node in nodes], -180, 180)
lat_range = padded_range([node['lat'] for node in nodes], -90, 90)
apply_geo_style(
    cluster_fig,
    f"OpenFlights Cluster {result['round']} with Intra-Cluster Routes ( $|S|$ ={res
    lon_range,
    lat_range,
)

cluster_path = os.path.join('generated', f"openflights_cluster_{result['round']}
cluster_fig.write_image(cluster_path, width=1200, height=700, scale=2)
display(Image(filename=cluster_path, width=1000))
print(f"Saved cluster map to {cluster_path}")

```



Saved overview map to generated\openflights_clusters_world_map.png



Saved cluster map to generated\openflights_cluster_1.png

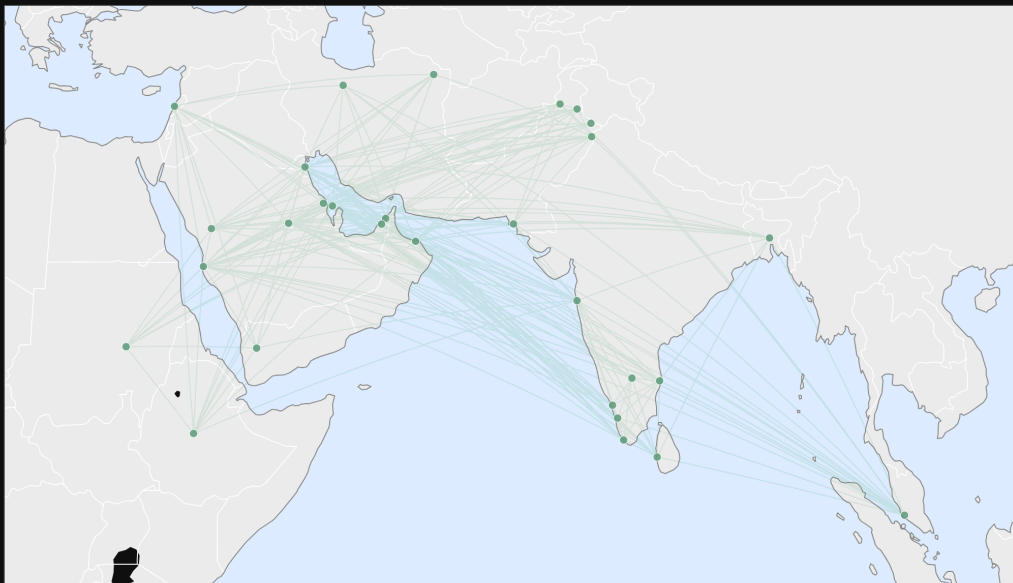
OpenFlights Cluster 2 with Intra-Cluster Routes ($|S|=53$, $|E[S]|=782$)



Cluster
● Round 2 airports

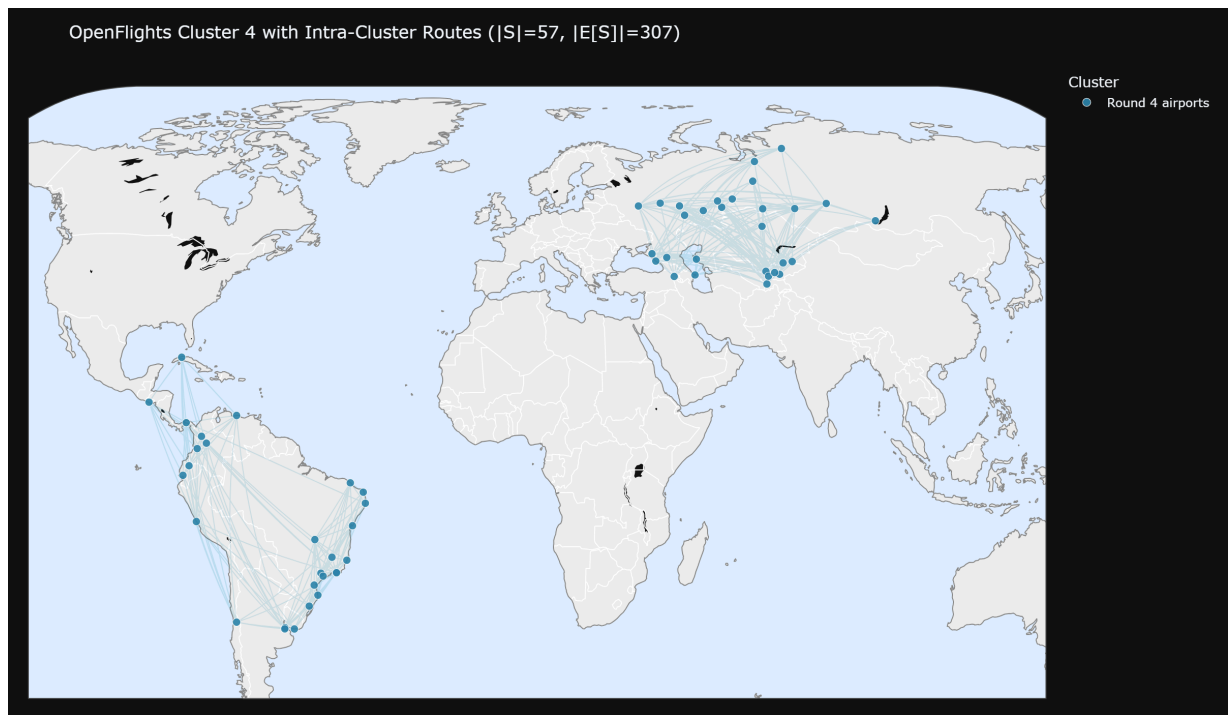
Saved cluster map to generated\openflights_cluster_2.png

OpenFlights Cluster 3 with Intra-Cluster Routes ($|S|=29$, $|E[S]|=217$)

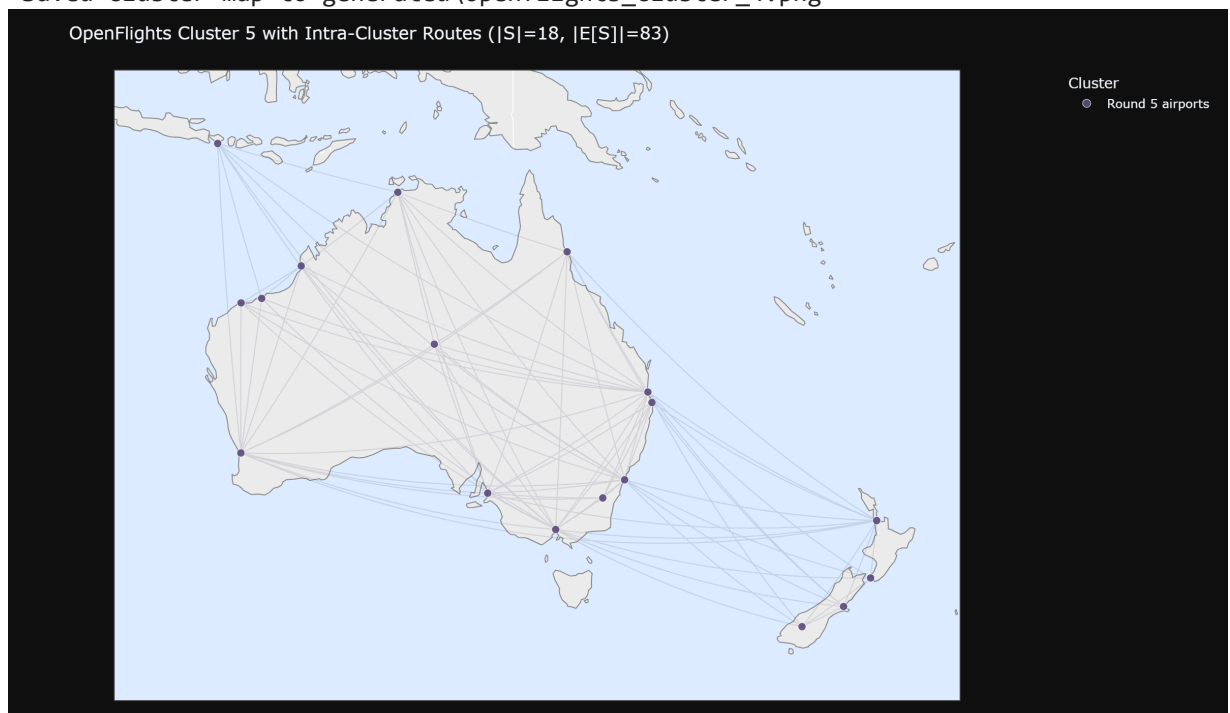


Cluster
● Round 3 airports

Saved cluster map to generated\openflights_cluster_3.png



Saved cluster map to generated\openflights_cluster_4.png



Saved cluster map to generated\openflights_cluster_5.png

Experiments for the Facebook Dataset

```
In [29]: import csv

facebook_disjoint_subgraphs = runDisjointSubgraphExtraction('data/musae_facebook_ed

for result in facebook_disjoint_subgraphs:
    print(f"Round {result['round']}:")
    print(f"|S| = {result['num_vertices']], density = {result['score']}\n")
```

```

with open('data/facebook_large/musae_facebook_target.csv', newline='', encoding='ut
    facebook_page_names = {
        int(row['id']): row['page_name']
        for row in csv.DictReader(f)
    }

facebook_densest_page_names = sorted(
    facebook_page_names[node_id]
    for node_id in facebook_disjoint_subgraphs[0]['vertex_set']
)

#for result in facebook_disjoint_subgraphs:
#    print(f"\n\nRound {result['round']}:")
#    print(sorted(facebook_page_names[node_id] for node_id in result['vertex_set']))

facebook_densest_page_names

```

Finished reading graph with 22470 nodes and 170823 edges.
Finished reading graph with 22091 nodes and 144246 edges.
Finished reading graph with 21994 nodes and 138591 edges.
Finished reading graph with 21232 nodes and 109432 edges.
Finished reading graph with 21168 nodes and 107522 edges.

Round 1:

|S| = 324, density = 35.0

Round 2:

|S| = 89, density = 25.359550561797754

Round 3:

|S| = 662, density = 20.358006042296072

Round 4:

|S| = 55, density = 17.345454545454544

Round 5:

|S| = 311, density = 15.434083601286174

['1-2 SBCT, 7th Infantry Division', '101st Airborne Division (Air Assault)', '101st Airborne Division Sustainment Brigade "Lifeliners"', '101st CAB, Wings of Destiny', '160th Signal Brigade', '16th Combat Aviation Brigade', '173rd Airborne Brigade', '1st Armored Brigade Combat Team, 3rd Infantry Division', '1st Armored Division', '1st Battalion, 506th Infantry Regiment "Red Currahee"', '1st Brigade Combat Team "Bastogne"', '1st Brigade Combat Team, 82nd Airborne Division', '1st Cavalry Division', '1st Combat Aviation Brigade, 1st Infantry Division', '1st Infantry Division', '1st Stryker Brigade Combat Team, 4th Infantry Division', '21st Theater Sustainment Command', '24: Legacy', '29th Infantry Division', '2nd Armored Brigade Combat Team, 1st Cavalry Division', '2nd Brigade Combat Team "STRIKE"', '2nd Brigade Combat Team, 3rd Infantry Division', '2nd Brigade Combat Team, 82nd Airborne Division', '2nd Infantry Brigade Combat Team, 4th Infantry Division', '2nd Stryker Brigade Combat Team, 2nd Infantry Division', '2nd Theater Signal Brigade', '3d U.S. Infantry Regiment (The Old Guard)', '3rd Armored Brigade Combat Team, 1st Armored Division', '3rd Brigade Combat Team, 1st Cavalry Division', '3rd Combat Aviation Brigade, 3rd Infantry Division (3 CAB, 3 ID)', '3rd Infantry Division', '4th Brigade Combat Team (Airborne), 25th Infantry Division', '4th Infantry Division', '5th Mobile Public Affairs Detachment', '7th Army Training Command', '7th Infantry Division', '82nd Airborne Division', '82nd Combat Aviation Brigade, 82nd Airborne Division', '85th Support Command', 'AFN Europe', 'AFOSR, Air Force Office of Scientific Research', 'APB FOX', 'Aberdeen Proving Ground', 'Aberdeen Proving Ground MWR', 'Adapting the Army', 'Air Combat Command', 'Air Force Special Operations Command', 'Air Mobility Command', 'Air National Guard Recruiting', 'Airman Magazine', 'Alabama National Guard', 'All Army Sports', 'Allied Land Command', 'American Battle Monuments Commission', 'American Grit', 'Arizona National Guard', 'Arkansas National Guard', 'Armament Research, Development and Engineering Center (ARDEC)', 'Armed Forces Entertainment', 'Armed Services Blood Program (ASBP)', 'Army Contracting Command, APG-Natick Contracting Division', 'Army Family and MWR Programs', 'Army Housing', 'Army Knowledge Online/Defense Knowledge Online', 'Army Medicine', 'Army OneSource', 'Army Study Program Management Office - ASPMO', 'Army Sustainment', 'Army Sustainment Command', 'Army Targeting Center', 'Army Training Network (ATN)', 'Army University Press', 'BOOM', 'BeThere Peer Support', 'Beat Shazam', 'Bob's Burgers', 'Bones', 'Bordertown', 'Brooklyn Nine-Nine', 'CAPE (Center for the Army Profession and Ethic)', 'CDC', 'CERDEC', 'California National Guard', 'Camp Atterbury', 'Chief of Air Defense and Regimental CSM', 'Colorado National Guard', 'Combined Joint Task Force-Horn of Africa', 'Computer/Electronic Accommodations Program (CAP)', 'Cooper Barrett's Guide to Surviving Life', 'Corpus Christi Army Depot',

'Coupled', 'Crane Army Ammunition Activity', 'DHA Immunization Healthcare Branch', 'DHS Science and Technology Directorate', 'Defense Advanced Research Projects Agency - DARPA', 'Defense Commissary Agency', 'Defense Finance and Accounting Service - DFA S', 'Defense Imagery', 'Defense Information Systems Agency', 'Defense Intelligence Agency', 'Defense Logistics Agency (DLA)', 'Defense TV', 'Department of Defense Education Activity (DoDEA)', 'Department of Homeland Security', 'Eighth Army-Korea', 'Empire', 'Employer Support of the Guard and Reserve', 'Europe District, U.S. Army Corps of Engineers', 'FEMA Federal Emergency Management Agency', 'FOX Teen Choice Awards', 'Family Guy', 'Famous', 'Fires Center of Excellence and Fort Sill', 'First Army Division East', 'Florida National Guard', 'Fort Bragg, N.C.', 'Fort George G. Meade, Md', 'Fort Gordon', 'Fort Knox Soldier for Life - Transition Assistance Program', 'Fort Leavenworth', 'Fort Riley', 'Fort Wainwright', 'Georgia National Guard', 'Ghosted', 'Glee', 'Gotham', 'GovX', 'Grandfathered', 'HQDA G-4', 'Headquarters Support Company, 603rd Aviation Support Battalion', 'Hell's Kitchen', 'Home Free FOX', 'Honolulu District, U.S. Army Corps of Engineers', 'Hotel Hell', 'Houdini & Doyle', 'I Corps', 'III Corps and Fort Hood', 'ISAF Joint Command (IJC)', 'Indiana Guardsman', 'JMR C', 'Joint Base Elmendorf-Richardson JBER (official)', 'Joint Base Lewis-McChord', 'Joint Base Myer-Henderson Hall Army Community Service (ACS)', 'Joint Services Support- Washington', 'Kansas National Guard', 'Kicking & Screaming', 'LA to Vegas', 'Let Hal Weapon FOX', 'Los Angeles District, U.S. Army Corps of Engineers', 'Louisiana National Guard', 'Love Connection FOX', 'Lucifer', 'Madigan Army Medical Center', 'Making History', 'Marine Corps Recruiting', 'Massachusetts National Guard - The Nation's First', 'MasterChef', 'MasterChef Junior', 'Medical Readiness and Training Command', 'Michigan National Guard', 'Military Health', 'Military OneSource', 'Mississippi National Guard', 'Missouri National Guard', 'Mobilization and Deployment, DPTMS', 'My Kitchen Rules', 'NASA - National Aeronautics and Space Administration', 'NATO', 'NORAD and USNORTHERN Command', 'National Center for PTSD - U.S. Department of Veterans Affairs', 'National Guard', 'National Guard Counterdrug Program', 'National Guard Patriot Academy', 'Navy Medicine', 'New Girl', 'New Jersey National Guard', 'New Year's Eve on FOX', 'New York Army National Guard', 'Norfolk District, U.S. Army Corps of Engineers', 'North Carolina National Guard', 'North Dakota National Guard', 'Office of the Army Chief Information Officer/G-6', 'Office of the Director of National Intelligence', 'Oklahoma National Guard', 'Operation Live Well', 'Operations Group, National Training Center', 'Oregon National Guard - Oregon Military Department', 'PEO C3T', 'PEO Soldier', 'Pacific Region, U.S. Army Installation Management Command', 'Party Over Here', 'Pennsylvania National Guard', 'Picatinny Arsenal', 'Pitch', 'Prison Break', 'Puerto Rico National Guard', 'RAF Lakenheath', 'RC-East', 'Resolute Support Mission', 'Returning Service Members - U.S. Dept. of Veterans Affairs', 'Rhode Island National Guard', 'Rock Island Arsenal', 'Rosewood', 'SDDC', 'STAR', 'Scream Queens', 'Second Chance', 'Shots Fired', 'Showtime At The Apollo', 'Sleepy Hollow', 'So You Think You Can Dance', 'Soldier for Life', 'Son of Zorn', 'Spouse Education and Career Opportunities Program - SECO', 'Superhuman', 'Supreme Headquarters Allied Powers Europe (SHAPE)', 'TRICARE', 'Team Redstone', 'Texas Army National Guard', 'The Connecticut National Guard', 'The Exorcist FOX', 'The F Word', 'The Fort Campbell Courier', 'The Gifted', 'The Grinder', 'The Joint Staff', 'The Last Man on Earth', 'The Mick', 'The National Guard', 'The Obama White House', 'The Orville', 'The Post-9/11 GI Bill, U.S. Department of Veterans Affairs', 'The Resident', 'The Simpsons', 'The United States Army Band', 'The White House', 'The X-Files', 'Tobyhanna Army Depot', 'Troops to Teachers', 'U.S. Africa Command (AFRICOM)', 'U.S. Air Force Medical Service - AFMS', 'U.S. Air Force Recruiting', 'U.S. Air Force Thunderbirds', 'U.S. Air Forces Central Command', 'U.S. Air Forces in Europe and Air Forces Africa', 'U.S. Army', 'U.S. Army AMIS', 'U.S. Army Acquisition Support Center', 'U.S. Army Africa', 'U.S. Army Armor School', 'U.S. Army Aviation and Missile Command', 'U.S. Army Aviation, Missile, Research, Development and Engineering Center', 'U.S. Army Cadet Command (ROTC)', 'U.S. Army Center of Military History', 'U.S. Army Central', 'U.S. Army Chaplain Corps', 'U.S. Army Civil Affairs & Psychological Operations Command (Airborne

e)', 'U.S. Army Combined Arms Center', 'U.S. Army Command and General Staff College', 'U.S. Army Corps of Engineers, Headquarters', 'U.S. Army Edgewood Chemical Biological Center - ECBC', 'U.S. Army Europe', 'U.S. Army Force Development Directorate', 'U.S. Army Fort Carson', 'U.S. Army Fort Drum & 10th Mountain Division', 'U.S. Army Fort Knox', 'U.S. Army Fort Lee', 'U.S. Army Future Soldier Family', 'U.S. Army Garrison Red Cloud', 'U.S. Army Garrison-Hawaii', 'U.S. Army John F. Kennedy Special Warfare Center and School', 'U.S. Army Materiel Command', 'U.S. Army Office of Small Business Programs', 'U.S. Army Office of the Chief of Public Affairs', 'U.S. Army Operational Test Command', 'U.S. Army Partnership for Youth Success (PaYS) Program', 'U.S. Army Public Affairs, Midwest', 'U.S. Army Public Health Center', 'U.S. Army Ready and Resilient', 'U.S. Army Research Laboratory', 'U.S. Army Research, Development and Engineering Command', 'U.S. Army Reserve', 'U.S. Army SHARP', 'U.S. Army School of Music', 'U.S. Army Sergeants Major Academy & The NCO Leadership Center of Excellence', 'U.S. Army Simulation & Training Technology Center', 'U.S. Army Space and Missile Defense Command (SMDC)', 'U.S. Army Training and Doctrine Command', 'U.S. Army Warrior Care and Transition', 'U.S. Army Women's Museum', 'U.S. Central Command', 'U.S. Coast Guard', 'U.S. Coast Guard Maritime Safety & Security Team - 91105', 'U.S. Department of State', 'U.S. Forces Korea', 'U.S. Marine Corps', 'U.S. Mission to NATO', 'U.S. Naval Forces Europe-Africa/U.S. Sixth Fleet', 'U.S. Navy', 'U.S. Pacific Air Forces', 'U.S. Pacific Command', 'U.S. Transportation Command (USTRANSCOM)', 'US Air Force Academy (Official)', 'US Army Combat Readiness Center', 'US Army Fort Benning', 'US Army Golden Knights', 'US Army Military District of Washington', 'US Army Recruiting Command (USAREC)', 'US National Archives', 'USA.gov', 'USACE and Fort Rucker', 'United States Air Force', 'Veterans Benefits Administration (VBA)', 'U.S. Department of Veterans Affairs', 'Virginia National Guard', 'Warrior Games', 'Washington National Guard', 'Wayward Pines', 'West Point - The U.S. Military Academy', 'Wisconsin National Guard', 'Womack Army Medical Center', 'World's Funniest', 'Wyoming National Guard', 'XVIII Airborne Corps', 'You The Jury']

Experiments for Optimal Quasi Clique

Now re-run the experiments from above with the new objective function and compare the results to the previous outputs.

Experiments for the OpenFlights Dataset

```
In [30]: openflights_tsourakakis_subgraphs = runDisjointSubgraphExtraction('data/openflights

for result in openflights_tsourakakis_subgraphs:
    density = objectiveValue(result['num_edges'], result['num_vertices'], objective
    clique_likeness = objectiveValue(result['num_edges'], result['num_vertices'], o
    print(f"Round {result['round']}: |S| = {result['num_vertices']}, density = {den

comparison_rows = [
    '| Round | Density: \\|S\\| | density | clique-like | \\| | Tsourakakis: \\|S\\|
    '|---:|---:|---:|---:|---:|---:|---:|'
]

for i in range(5):
    density_result = openflights_disjoint_subgraphs[i]
    density_density = objectiveValue(density_result['num_edges'], density_result['n
    density_clique = objectiveValue(density_result['num_edges'], density_result['nu
```

```

tsourakakis_result = openflights_tsourakakis_subgraphs[i]
tsourakakis_density = objectiveValue(tsourakakis_result['num_edges'], tsourakakis_result['num_vertices'])
tsourakakis_clique = objectiveValue(tsourakakis_result['num_edges'], tsourakakis_result['num_vertices'])
comparison_rows.append(
    f"| {i + 1} | {density_result['num_vertices']} | {density_density:.4f} | {density_clique:.4f} |"
)

display(Markdown('\n'.join(comparison_rows)))

```

Finished reading graph with 3214 nodes and 18858 edges.
 Finished reading graph with 2989 nodes and 11956 edges.
 Finished reading graph with 2755 nodes and 8812 edges.
 Finished reading graph with 2617 nodes and 6492 edges.
 Finished reading graph with 2515 nodes and 5647 edges.
 Round 1: |S| = 93, density = 24.2688, clique-likeness = 0.5276
 Round 2: |S| = 45, density = 18.1333, clique-likeness = 0.8242
 Round 3: |S| = 55, density = 16.9273, clique-likeness = 0.6269
 Round 4: |S| = 29, density = 8.5517, clique-likeness = 0.6108
 Round 5: |S| = 22, density = 5.7727, clique-likeness = 0.5498

Round	Density: S	density	clique-like		Tsourakakis: S	density	clique-like
1	180	25.3778	0.2836		93	24.2688	0.5276
2	53	14.7547	0.5675		45	18.1333	0.8242
3	29	7.4828	0.5345		55	16.9273	0.6269
4	57	5.3860	0.1924		29	8.5517	0.6108
5	18	4.6111	0.5425		22	5.7727	0.5498

AI-Assisted OpenFlights Visualization for the Tsourakakis Objective

The following cell uses the same map style as above and visualizes the five OpenFlights clusters found with the Tsourakakis objective.

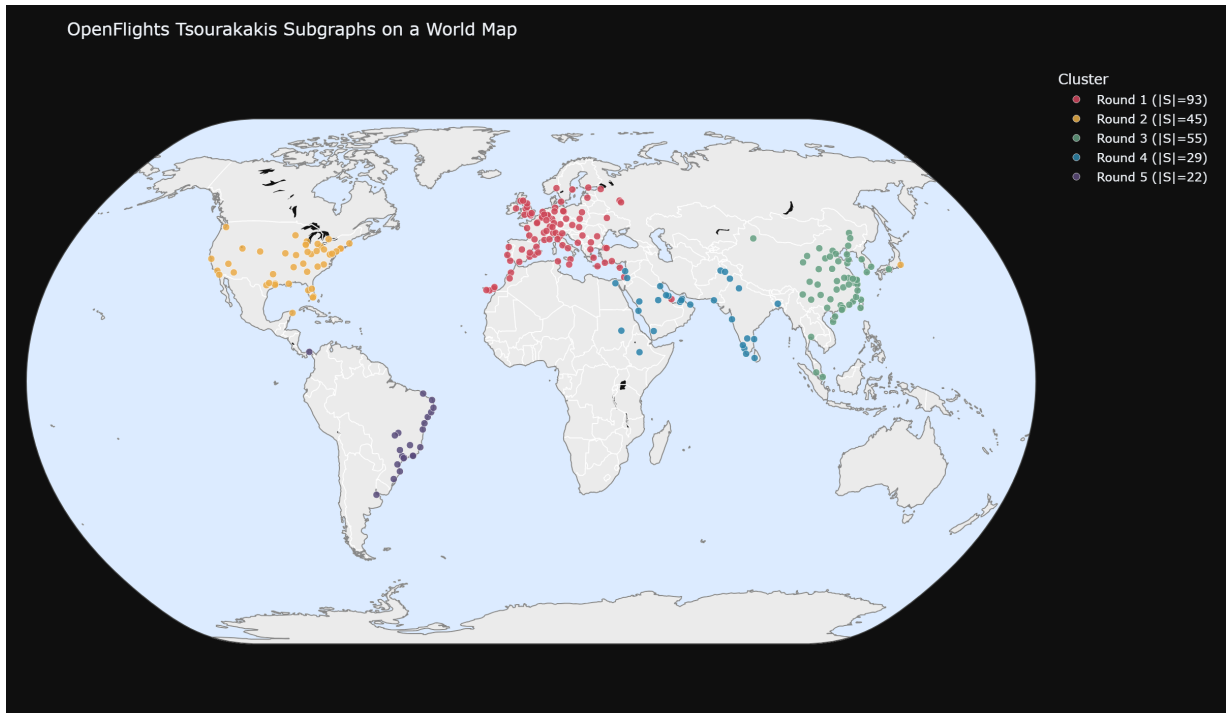
```

In [31]: tsourakakis_overview_fig = go.Figure()
for result, color in zip(openflights_tsourakakis_subgraphs, cluster_colors):
    _, nodes = cluster_nodes(result['vertex_set'])
    tsourakakis_overview_fig.add_trace(go.Scattergeo(
        lon=[node['lon'] for node in nodes],
        lat=[node['lat'] for node in nodes],
        text=[airport_hover_text(result['round'], node) for node in nodes],
        mode='markers',
        marker=dict(size=7, color=color, opacity=0.85, line=dict(width=0.5, color='black')),
        name=f"Round {result['round']} (|S|={result['num_vertices']})",
        hovertemplate="%{text}<extra></extra>",
    ))

apply_geo_style(tsourakakis_overview_fig, 'OpenFlights Tsourakakis Subgraphs on a World Map')
tsourakakis_overview_path = os.path.join('generated', 'openflights_tsourakakis_world_map.html')
tsourakakis_overview_fig.write_image(tsourakakis_overview_path, width=1200, height=800)

```

```
display(Image(filename=tsourakakis_overview_path, width=1000))
print(f"Saved Tsourakakis overview map to {tsourakakis_overview_path}")
```



Saved Tsourakakis overview map to generated\openflights_tsourakakis_world_map.png

Experiments for the Facebook Dataset

```
In [32]: facebook_tsourakakis_subgraphs = runDisjointSubgraphExtraction('data/musae_facebook

for result in facebook_tsourakakis_subgraphs:
    density = objectiveValue(result['num_edges'], result['num_vertices'], objective
    clique_likeness = objectiveValue(result['num_edges'], result['num_vertices'], o
    print(f"Round {result['round']}: |S| = {result['num_vertices']}, density = {den

comparison_rows = [
    '| Round | Density: \\|S\\| | density | clique-like | \\| | Tsourakakis: \\|S\\|
    '|---:|---:|---:|---:|---:|---:|---:|'
]

for i in range(5):
    density_result = facebook_disjoint_subgraphs[i]
    density_density = objectiveValue(density_result['num_edges'], density_result['n
    density_clique = objectiveValue(density_result['num_edges'], density_result['nu

    tsourakakis_result = facebook_tsourakakis_subgraphs[i]
    tsourakakis_density = objectiveValue(tsourakakis_result['num_edges'], tsourakak
    tsourakakis_clique = objectiveValue(tsourakakis_result['num_edges'], tsourakaki
    comparison_rows.append(
        f"| {i + 1} | {density_result['num_vertices']} | {density_density:.4f} | {d
    )

display(Markdown('\n'.join(comparison_rows)))
```

```

for result in facebook_tsourakakis_subgraphs:
    print(f"\n\nRound {result['round']}:")
    print(sorted(facebook_page_names[node_id] for node_id in result['vertex_set']))

```

Finished reading graph with 22470 nodes and 170823 edges.

Finished reading graph with 22408 nodes and 168565 edges.

Finished reading graph with 22251 nodes and 150089 edges.

Finished reading graph with 22154 nodes and 144415 edges.

Finished reading graph with 22111 nodes and 143463 edges.

Round 1: $|S| = 60$, density = 29.3167, clique-likeness = 0.9938

Round 2: $|S| = 129$, density = 33.3566, clique-likeness = 0.5212

Round 3: $|S| = 89$, density = 25.3596, clique-likeness = 0.5764

Round 4: $|S| = 38$, density = 17.6579, clique-likeness = 0.9545

Round 5: $|S| = 41$, density = 17.7073, clique-likeness = 0.8854

Round	Density: $ S $	density	clique-like		Tsourakakis: $ S $	density	clique-like
1	324	35.0000	0.2167		60	29.3167	0.9938
2	89	25.3596	0.5764		129	33.3566	0.5212
3	662	20.3580	0.0616		89	25.3596	0.5764
4	55	17.3455	0.6424		38	17.6579	0.9545
5	311	15.4341	0.0996		41	17.7073	0.8854

Round 1:

['24: Legacy', 'APB FOX', 'American Grit', 'BOOM', 'Beat Shazam', 'Bob's Burgers', 'Bones', 'Bordertown', 'Brooklyn Nine-Nine', 'Cooper Barrett's Guide to Surviving Life', 'Coupled', 'Empire', 'FOX Teen Choice Awards', 'Family Guy', 'Famous', 'Ghosted', 'Glee', 'Gotham', 'Grandfathered', 'Hell's Kitchen', 'Home Free FOX', 'Hotel Hell', 'Houdini & Doyle', 'Kicking & Screaming', 'LA to Vegas', 'Lethal Weapon FOX', 'Love Connection FOX', 'Lucifer', 'Making History', 'MasterChef', 'MasterChef Junior', 'My Kitchen Rules', 'New Girl', 'New Year's Eve on FOX', 'Party Over Here', 'Pitch', 'Prison Break', 'Rosewood', 'STAR', 'Scream Queens', 'Second Chance', 'Shots Fired', 'Showtime At The Apollo', 'Sleepy Hollow', 'So You Think You Can Dance', 'Son of Zorn', 'Superhuman', 'The Exorcist FOX', 'The F Word', 'The Gifted', 'The Grinder', 'The Last Man on Earth', 'The Mick', 'The Orville', 'The Resident', 'The Simpsons', 'The X-Files', 'Wayward Pines', 'World's Funniest', 'You The Jury']

Round 2:

['1-2 SBCT, 7th Infantry Division', '101st Airborne Division (Air Assault)', '16th Combat Aviation Brigade', '173rd Airborne Brigade', '1st Armored Division', '1st Battalion, 506th Infantry Regiment "Red Currahee"', '1st Brigade Combat Team, 82nd Airborne Division', '1st Cavalry Division', '1st Combat Aviation Brigade, 1st Infantry Division', '1st Infantry Division', '2nd Theater Signal Brigade', '3d U.S. Infantry Regiment (The Old Guard)', '3rd Combat Aviation Brigade, 3rd Infantry Division (3 CAB, 3 ID)', '3rd Infantry Division', '4th Infantry Division', '7th Infantry Division', '82nd Airborne Division', '82nd Combat Aviation Brigade, 82nd Airborne Division', 'AFOSR, Air Force Office of Scientific Research', 'Aberdeen Proving Ground', 'Adapting the Army', 'American Battle Monuments Commission', 'Armament Research, Development and Engineering Center (ARDEC)', 'Army Contracting Command, APG-Natick Contracting Division', 'Army Family and MWR Programs', 'Army Housing', 'Army Knowledge Online/Defense Knowledge Online', 'Army Medicine', 'Army OneSource', 'Army Study Program Management Office - ASPMO', 'Army Sustainment', 'Army Training Network (ATN)', 'Army University Press', 'CAPE (Center for the Army Profession and Ethic)', 'CERDEC', 'Corpus Christi Army Depot', 'DHA Immunization Healthcare Branch', 'Defense Advanced Research Projects Agency - DARPA', 'Defense Commissary Agency', 'Defense Finance and Accounting Service - DFAS', 'Defense Information Systems Agency', 'Defense Logistics Agency (DLA)', 'Defense TV', 'Department of Defense Education Activity (DoDEA)', 'Europe District, U.S. Army Corps of Engineers', 'FEMA Federal Emergency Management Agency', 'Fires Center of Excellence and Fort Sill', 'Fort Bragg, N.C.', 'Fort George G. Meade, Md', 'Fort Gordon', 'Fort Riley', 'Fort Wainwright', 'HQDA G-4', 'Honolulu District, U.S. Army Corps of Engineers', 'I Corps', 'III Corps and Fort Hood', 'Joint Base Lewis-McChord', 'Joint Services Support- Washington', 'Los Angeles District, U.S. Army Corps of Engineers', 'Marine Corps Recruiting', 'Military Health', 'Military OneSource', 'NORAD and USNORTHERN Command', 'National Guard', 'Norfolk District, U.S. Army Corps of Engineers', 'Operations Group, National Training Center', 'PEO C3T', 'PEO Soldier', 'Pacific Region, U.S. Army Installation Management Command', 'Picatinny Arsenal', 'Resolute Support Mission', 'Rock Island Arsenal', 'Soldier for Life', 'TRICARE', 'Texas Army National Guard', 'The Joint Staff', 'The National Guard', 'The Obama White House', 'The White House', 'Tobyhanna Army Depot', 'Troops to Teachers', 'U.S. Africa Command (AFRICOM)', 'U.S. Army', 'U.S. Army Acquisition Support Center', 'U.S. Army Africa', 'U.S. Army Armor School', 'U.S. Army Aviation and Missile Command', 'U.S. Army Cadet Command (ROTC)', 'U.S. Army Center of Military History', 'U.S. Army Central', 'U.S. Army Chaplain Corps', 'U.S. Army Combined Arms Center', 'U.S. Army Command and General Staff College', 'U.S. Army Corps of Engineers, Headquarters', 'U.S. Army Europe', 'U.S. Army Force Development Directorate', 'U.S. Army Fort Carson', 'U.S. Army Fort Drum & 10th Mountain Division', 'U.S. Army Garrison Red Cloud', 'U.S. Army Garrison-Hawaii', 'U.S. Army Materiel Command', 'U.S. Army Office

of the Chief of Public Affairs', 'U.S. Army Partnership for Youth Success (PaYS) Program', 'U.S. Army Public Affairs, Midwest', 'U.S. Army Ready and Resilient', 'U.S. Army Research Laboratory', 'U.S. Army Research, Development and Engineering Command', 'U.S. Army Reserve', 'U.S. Army SHARP', 'U.S. Army Simulation & Training Technology Center', 'U.S. Army Space and Missile Defense Command (SMDC)', 'U.S. Army Training and Doctrine Command', 'U.S. Army Warrior Care and Transition', 'U.S. Army Women's Museum', 'U.S. Central Command', 'U.S. Coast Guard', 'U.S. Forces Korea', 'U.S. Marine Corps', 'U.S. Navy', 'U.S. Pacific Command', 'U.S. Transportation Command (USTRANSCOM)', 'US Army Combat Readiness Center', 'US Army Fort Benning', 'US Army Golden Knights', 'US Army Military District of Washington', 'US Army Recruiting Command (USAREC)', 'United States Air Force', 'West Point - The U.S. Military Academy', 'XVIII Airborne Corps']

Round 3:

['Comisión Europea en España', 'Commission Européenne en Belgique / Europese Commissie in België', 'Commission européenne au Luxembourg', 'Commission européenne en France', 'Council of Europe', 'Council of the European Union', 'Delegation of the European Union to Japan', 'Delegation of the European Union to Libya بعثة الاتحاد الأوروبي في ليبيا', 'Digital Single Market', 'EESC - European Economic and Social Committee', 'EU Budget', 'EU Delegation to the Council of Europe', 'EU Environment', 'EU Internal Market, Industry, Entrepreneurship & SMEs', 'EU Interpreters', 'EU Justice and Consumers', 'EU Law and Publications', 'EU Neighbourhood & Enlargement', 'EU Science & Innovation', 'EU Science Hub - Joint Research Centre', 'EU in India', 'EU in South Africa', 'EU to Barbados, Eastern Caribbean States, the OECS, Caricom & Cariforum', 'EU-kommissionen i Sverige', 'Euroopa Komisjoni esindus Eestis', 'Euroopan komissio Suomessa - Europeiska kommissionen i Finland', 'European Commission', 'European Commission - Civil Protection & Humanitarian Aid Operations - ECHO', 'European Commission - Development & Cooperation - EuropeAid', 'European Commission - Economy and Finance', 'European Commission Representation in Ireland', 'European Commission in the UK', 'European External Action Service - EEAS', 'European Parliament', 'European Union at the UN - Geneva', 'European Union at the UN - New York', 'European Union in Albania', 'European Union in Armenia', 'European Union in Bangladesh', 'European Union in Belarus', 'European Union in Canada', 'European Union in Egypt', 'European Union in Hong Kong and Macao', 'European Union in Indonesia', 'European Union in Israel', 'European Union in Korea', 'European Union in Lebanon', 'European Union in Malaysia', 'European Union in Mauritius', 'European Union in New Zealand', 'European Union in Russia', 'European Union in Switzerland and Liechtenstein', 'European Union in Tajikistan', 'European Union in Thailand', 'European Union in The Gambia', 'European Union in Turkey', 'European Union in Ukraine', 'European Union in Uzbekistan', 'European Union in Vietnam', 'European Union in the Philippines', 'European Union in the Republic of Moldova', 'Europäische Kommission - Vertretung in Österreich', 'Europäische Kommission - Vertretung in Deutschland', 'Evropska komisija v Sloveniji', 'Evropska unija u Bosni i Hercegovini', 'Federica Mogherini', 'Frans Timmermans', 'Jean-Claude Juncker', 'Jyrki Katainen', 'Komisja Europejska w Polsce', 'Marianne Thyssen', 'Representació de la Comissió Europea a Barcelona', 'Representação da Comissão Europeia em Portugal', 'Social Europe', 'Tanja Fajon', 'Union Européenne à Djibouti', 'Union européenne au Cameroun', 'Union européenne au Maroc', 'Union européenne en Côte d'Ivoire', 'União Europeia na Guiné-Bissau', 'União Europeia no Brasil', 'Unión Europea en Bolivia / European Union in Bolivia', 'Unión Europea en Chile', 'Unión Europea en El Salvador', 'Unión Europea en Perú', 'Unión Europea en Republica Dominicana', 'Vytenis Andriukaitis', 'Ευρωπαϊκή Επιτροπή στην Ελλάδα', 'Европейската комисия в България / European Commission in Bulgaria']

Round 4:

['Cultural Heritage Administration of Korea', 'Defense Acquisition Program Administration', 'Facebook', 'Ministry of Health and Welfare, Republic of Korea', 'Ministry of Science and ICT', '대한민국 고용노동부', '대한민국 공정거래위원회', '대한민국 관세청', '대한민국 국가보훈처', '대한민국 국무조정실·국무총리비서실', '대한민국 국민권익위원회', '대한민국 국방부', '대한민국 국세청', '대한민국 국토교통부', '대한민국 금융위원회', '대한민국 기상청 Korea Meteorological Administration', '대한민국 기획재정부', '대한민국 농림축산식품부', '대한민국 농촌진흥청', '대한민국 문화체육관광부', '대한민국 방송통신위원회', '대한민국 법무부(Ministry of Justice)', '대한민국 법제처', '대한민국 병무청', '대한민국 산림청', '대한민국 산업통상자원부 '경제다반사'', '대한민국 소방청', '대한민국 여성가족부 "가족사랑"', '대한민국 외교부', '대한민국 정부', '대한민국 조달청', '대한민국 통계청', '대한민국 통일부', '대한민국 특허청', '대한민국 해양수산부', '대한민국 행정안전부', '대한민국 행정중심복합도시건설청', '대한민국 환경부']

Round 5:

['Amanda Rishworth MP', 'Andrew Giles MP', 'Andrew Leigh MP', 'Anna Burke', 'Anthony Albanese MP', 'Bill Shorten MP', 'Brendan O'Connor MP', 'Catherine King', 'Cathy O'Toole MP', 'Chris Bowen', 'Clare O'Neil MP', 'Doug Cameron', 'Ed Husic MP', 'Jason Clare', 'Jenny Macklin MP', 'Jess Pugh for Mount Ommaney', 'Jim Chalmers MP - Member for Rankin', 'Joanne Ryan MP', 'Joel Fitzgibbon MP', 'Julia Gillard', 'Julie Collins MP', 'Julie Owens MP', 'Kate Ellis MP', 'Mark Butler MP', 'Mark Dreyfus', 'Michelle Rowland MP', 'Penny Wong - Senator for SA', 'Peta Murphy', 'Richard Marles MP', 'Rob Mitchell MP - Federal Labor Member for McEwen', 'Senator Carol Brown', 'Senator Glenn Sterle', 'Senator Helen Polley', 'Sharon Claydon MP', 'Shayne Neumann MP', 'Stephen Jones MP', 'Tanya Plibersek', 'Terri Butler MP', 'Tim Watts MP', 'Tony Burke MP', 'Wayne Swan']

How do the results differ? The Tsourakakis objective generally returns smaller but more clique-like subgraphs than the density objective. The effect is visible on OpenFlights and much stronger on Facebook, where the density objective often keeps larger but looser groups, while the Tsourakakis objective favors tighter communities.